Pattern Pin

A hybrid of pattern unlock and PIN unlock

Specification

Abstract

Pattern Pin is a device unlock that keeps a remembered gesture and changes the symbols under that gesture every time the lock is shown. At enrollment the owner draws a pattern on a square grid of single symbols. The ordered symbols under the pattern are the pin contents. Those contents generate the private key, and the private key stays a seed in hardware-backed storage. Each presentation of the lock is a lock instance with its own nonce. The nonce lives only as long as that instance. The private key and the nonce expand into a new public pin sequence, the sequence is laid onto the stored pattern, and the hash of the sequence is that instance's public hash. The owner repeats the gesture. The phone hashes the symbols the gesture reads and accepts the attempt when the digest equals the public hash. A match authorizes one use of the private key to unwrap the device key. Turning the display off, dismissing the lock, or completing a successful unlock wipes the nonce, the public sequence, the public hash, and the grid. The next presentation generates a different public hash from the same private key.

Contents

1 Introduction **2**

2 Terms **2**

3 Alphabet and canonical bytes **3**

4 The pattern **3**

5 Enrollment **3**

6 What is stored **4**

7 Lock instance **5**

8 Layout of the public grid **5**

9 Verification **6**

10 Release, wipe, and reset **7**

11 Attempt policy **7**

12 Worked illustration **7**

13 Threat model **8**

14 Parameter summary **10**

15 Boundaries **10**

1 Introduction

A phone today offers two familiar unlocks. A pattern unlock asks the owner to repeat a path on a fixed grid. A PIN unlock asks the owner to repeat a digit string on a fixed keypad. Pattern Pin is the hybrid of the two. The owner remembers a path. The device remembers the symbols that path crossed when the lock was created, and it uses those symbols as the seed of a private key. Every later presentation builds a new public pin sequence from that private key and lays the sequence onto the same path. Repeating the path reads the public sequence. The hash of that reading is what the phone checks.

The owner does not memorize the symbols. The symbols at enrollment are a random covering, one per cell, used once to create the pin contents. The symbols at unlock are a fresh covering, generated for that lock instance, and they are discarded with the instance. What remains between presentations is the sealed record: the pin contents, the layout pattern, and a device key wrapped by the private key.

2 Terms

Grid A square of n cells, where n is 9 or 16. A 9-cell grid has three rows and three columns. A 16-cell grid has four rows and four columns. Cells are numbered from 1 in row-major order, left to right, top to bottom.

Symbol One character from the alphabet in Section 3. Each cell displays exactly one symbol.

Pattern The ordered sequence of distinct cells produced by one continuous stroke, after the pass-over rule in Section 4. Stored, this sequence is the layout pattern. It places public symbols. It is not the input to the private key.

Pin contents The ordered symbols that lay under the pattern at enrollment. This string is the material that generates the private key.

Private key A 32-byte seed derived from the pin contents. It is used to expand each public sequence and to unwrap the device key. It is not displayed and it is not written into the public grid.

Lock instance One presentation of the lock, from the moment the grid is shown until the instance is wiped.

Nonce A fresh 32-byte value that exists only inside one lock instance.

Public sequence The ordered pin symbols expanded from the private key and the nonce. Its length is the length of the layout pattern. These are the public pins for that instance.

Public hash The SHA-256 digest of the canonical encoding of the public sequence. A different nonce produces a different public sequence and therefore a different public hash.

Readout The ordered symbols on the cells of a stroke the owner has just drawn, interpreted on the grid that is currently shown.

Device key A random 32-byte key created at enrollment and wrapped under the private key. The operating system uses it to unlock the device. The private key is the seed that unwraps it.

3 Alphabet and canonical bytes

The alphabet has 110 symbols. Indexes are a single byte.

Indexes	Count	Symbols
0–9	10	Digits `0` through `9`
10–35	26	Latin capitals `A` through `Z`
36–61	26	Latin small letters `a` through `z`
62–85	24	Greek capitals from `U+0391` through `U+03A9`, in code-point order, excluding the unassigned code point `U+03A2`
86–109	24	Greek small letters from `U+03B1` through `U+03C9`, in code-point order, excluding final sigma `U+03C2`

The canonical encoding of an ordered symbol string is the concatenation of these indexes, one byte each, with no separator and no length prefix. Enrollment, public-sequence generation, and verification all use this encoding. A stroke of a different length is a different byte string and yields a different digest.

4 The pattern

The owner chooses $n \in \{9, 16\}$ at the start of enrollment. That choice is fixed for the life of the sealed record.

A stroke is a continuous pointer path across cell centers. The path is reduced to a pattern as follows.

1. Give each cell integer coordinates. Column is $(c-1) \bmod \sqrt{n}$, and row is $\lfloor (c-1)/\sqrt{n} \rfloor$, with row 0 at the top.
2. From the current cell $A = (x_1, y_1)$ to the cell $B = (x_2, y_2)$ where the pointer next arrives, let $d_x = x_2 - x_1$, $d_y = y_2 - y_1$, and let $g = \gcd(|d_x|, |d_y|)$. If $A = B$, ignore the step.
3. For $i = 1, \ldots, g$, visit the lattice point $A + (id_x/g,\ id_y/g)$. Append the cell at that point when it is not already in the pattern.
4. The finished pattern is the append order. A pattern of fewer than four distinct cells is rejected. A pattern longer than n cannot occur, because cells are not repeated.

The pass-over rule is the familiar one from a phone pattern lock: a straight step that crosses a cell center includes that cell. An already included center is not added again, and the stroke may continue across it.

5 Enrollment

Enrollment builds the sealed record. It does not build a reusable public grid.

1. Draw n distinct symbols from the alphabet without replacement, using an unbiased sample from the platform CSPRNG, and place them on the n cells. This covering is random. It is not a function of a key.

2. The owner draws a pattern. The candidate pin contents are the symbols under that pattern, in pattern order.

3. The owner draws the pattern again on the same covering. The second canonical string must equal the first. If it does not, discard the pair and collect a new pair on the same covering. If enrollment is abandoned, wipe the covering and store nothing.

4. On a confirmed pair, form the record:

   (a) Store the layout pattern, the ordered cell numbers.

   (b) Store the pin contents, the ordered symbol indexes.

   (c) Draw a 16-byte salt from the CSPRNG.

   (d) Derive the private key as Argon2id over the canonical pin contents, with that salt, memory cost 65536 KiB, 3 iterations, parallelism 4, version `0x13`, and a 32-byte output. These are the low-memory parameters of RFC 9106, and they are stored in the record so verification uses the same cost.

   (e) Draw a 32-byte device key from the CSPRNG. Wrap it with AES-256-GCM under the private key. The GCM nonce is 12 random bytes. The additional data is the byte string defined below. Store the nonce, the 32-byte ciphertext, and the 16-byte tag.

   (f) Set the failure counter to 0.

   (g) Wipe the enrollment covering, the plaintext device key, and the working copy of the private key.

The additional data for the wrap, and the identity of the record, is this concatenation:

Field	Width
Version, value 1	1 byte
Grid size n	1 byte
Salt	16 bytes
Pattern length k	1 byte
Cell numbers of the layout pattern, in order	k bytes
Alphabet indexes of the pin contents, in order	k bytes

Integer fields that are wider than one byte are big-endian: the Argon2 memory, iteration, and parallelism parameters, the Argon2 version, and the failure counter.

The sealed record lives in hardware-backed storage. The lock interface can ask that storage to run the operations in this paper. Ordinary applications do not receive the pin contents, the private key, the salt, the layout pattern, or the unwrapped device key.

6 What is stored

Two long-term values sit beside each other, and they have different jobs.

The pin contents are the secret material. They are the ordered symbols from enrollment, and they are the Argon2id input that generates the private key. Cell coordinates are not that input. A copy of the private key does not have to be stored between instances: the trusted path derives it again from the sealed contents and the salt when an instance starts, and it clears the working copy when the instance ends.

The layout pattern is the stored gesture. It is required to build the next grid. The private key and the layout pattern together place the newly created public sequence: the key produces the symbols, and the pattern decides which cells receive them, in order. The phone never treats a match of cell numbers as success. Success is a match of hashes, as Section 9 specifies.

The rest of the long-term record is the salt, the Argon2 parameters, the wrapped device key, the grid size, the version, and the failure counter. The enrollment covering is not kept.

7 Lock instance

A lock instance begins when the device needs to show the lock and the attempt policy in Section 11 allows a new instance. It ends when the display is turned off, when the lock is dismissed, or when a successful match has released the device key. While the display stays on and the attempt has not succeeded, the same instance, the same nonce, and the same public hash remain in place, including across a wrong stroke.

1. Draw a 32-byte nonce from the CSPRNG. Hold it in the instance cache only.
2. Derive the private key from the sealed pin contents and salt. Hold it in trusted working memory for the instance.
3. Expand a symbol stream with HKDF-SHA-256, as in RFC 5869. The input keying material is the private key. The salt is the nonce. The info string is the ASCII bytes of `pattern-pin/public-grid/v1`. Expansion may request further blocks up to the HKDF limit. The stream is consumed in order and is deterministic for a given private key and nonce.
4. Turn the stream into unbiased alphabet indexes. For each byte b, if $b < 220$, emit index $b \bmod 110$; otherwise discard b. The bound $220 = 2 \times 110$ removes the bias from a 256-value byte.
5. Take the first k emitted indexes, duplicates allowed, as the public sequence. Set the public hash to SHA-256 over that canonical byte string.
6. Lay out the grid as in Section 8.
7. Show the grid. The public hash, the nonce, the private key, and the public sequence remain in the trusted path. They are not printed as themselves. The owner sees symbols.

Turning the display off destroys this nonce. The next time the lock is shown, a new nonce is drawn and the expansion is run again. The public sequence changes, and the public hash changes with it. A hash captured from an earlier instance has no instance left to match.

8 Layout of the public grid

The grid of a lock instance is a function of the private key, the nonce, and the stored layout pattern. There is no second random source.

1. Write the public sequence onto the layout pattern in order. The first public symbol is placed on the first cell of the pattern, the second symbol on the second cell, and so on through the k cells.

2. Let S be the set of distinct indexes in the public sequence.

3. Walk every cell that is not on the layout pattern, in row-major order. Continue the same symbol stream. The next emitted index that is not in S becomes that cell's decoy. The exclusion keeps a decoy from copying a symbol that the public sequence uses, so a stray cell does not display a public symbol.

4. The resulting covering is the grid that is shown.

Because at most 16 cells exist and the alphabet has 110 symbols, the decoy walk always has a symbol left to draw. The walk is deterministic, so the whole grid is determined by the private key, the nonce, and the pattern.

The public sequence is the part the gesture is meant to read. The decoys are the rest of the square. Both come from the expansion, which is why the grid belongs to the public side of that instance and can be wiped without touching the sealed pin contents.

9 Verification

The phone checks one digest: the hash of the full readout of the stroke the owner just drew.

1. Interpret the pointer path with the rule in Section 4, on the grid this instance is showing. Reject a result shorter than four cells.

2. Read the symbol now displayed on each cell of that stroke, in the stroke's order. This readout is taken from the presented grid and from the stroke that was drawn. The verifier does not substitute the stored layout pattern and then hash that.

3. Compute SHA-256 over the canonical readout.

4. Compare that digest with the instance's public hash, in constant time, as two 32-byte strings.

5. If they are equal, the stroke produced the public sequence for this nonce. Release the device key as in Section 10.

6. If they differ, leave the sealed record unchanged, increment the failure counter, and apply the attempt policy. The private key is not released. While the display stays on, the same public hash remains the check value.

A correct repeated pattern produces this match because the layout wrote the public sequence on that pattern. The hash of the readout is then the hash that was stored for the instance when the sequence was generated. That is the sense in which the public hash is generated from the private key: the sequence is an expansion of the private key and the nonce, and the public hash is the digest of the sequence. The construction starts from the sequence and hashes it. It does not start from a digest and search for a symbol string, which a cryptographic hash would not allow.

10 Release, wipe, and reset

A matching public hash authorizes one use of the private key. The use is the unwrap. The trusted path decrypts the AES-256-GCM blob with the private key and the additional data of the sealed record, checks the tag, and passes the plaintext device key to the operating system's unlock function. The private key itself is not passed out, is not shown, and is not placed in the instance cache as a public value.

The instance cache is then wiped. Wiping overwrites, with zeros, the nonce, the public sequence, the public hash, the symbol stream, the presented grid, the working private key, and the plaintext device key in the verifier. The sealed pin contents, the layout pattern, the salt, and the wrapped device key stay. The failure counter is set to 0. The system is reset: the next time a lock is required, Section 7 starts over, draws a new nonce, and generates a different public hash from the same private key.

The same wipe runs when the display is turned off and when the lock is dismissed, except the failure counter is left as it is and the device key is not unwrapped. Shutting the screen off is enough to make the next attempt a new public hash.

11 Attempt policy

The failure counter is part of the sealed record. Turning the display off does not clear it. Only a successful release sets it to 0.

Let c be the counter after a failed comparison. A new stroke in the live instance, or the minting of a nonce for the next instance, waits d seconds:

$$d = \begin{cases} 0 & \text{if } c < 5, \\ \min\bigl(3600,\ 30 \times 2^{c-5}\bigr) & \text{if } c \geq 5. \end{cases}$$

Five failures are free of delay. The sixth waits 30 seconds, and the wait doubles from there until it reaches one hour. The public hash does not change merely because the wait started. It changes when the instance is wiped and a later instance mints a new nonce.

12 Worked illustration

This section uses a 3×3 grid and a five-cell pattern so the bytes can be checked by hand. The enrollment covering is an illustration of a random placement. The unlock covering is an illustration of a placement the expansion might have produced. A deployed phone replaces both with a real sample and a real HKDF output. The digest in the unlock half is the real SHA-256 of the illustrated public sequence.

The pattern is cells $1, 2, 5, 9, 8$. Each step is a king-move to a neighboring cell, so the pass-over rule adds nothing further.

The enrollment readout $\kappa, 7, \Delta, \mathrm{R}, \omega$ has indexes $95, 7, 65, 27, 109$, which are the bytes

```
5f 07 41 1b 6d.
```

Argon2id over that byte string, with the record's salt and the parameters in Section 5, is the private key. The enrollment covering is then wiped.

Suppose the next lock instance expands to the public sequence $4, \sigma, \mathrm{a}, \mathrm{Z}, 9$. Those indexes are $4, 103, 36, 35, 9$, the bytes

```
04 67 24 23 09.
```

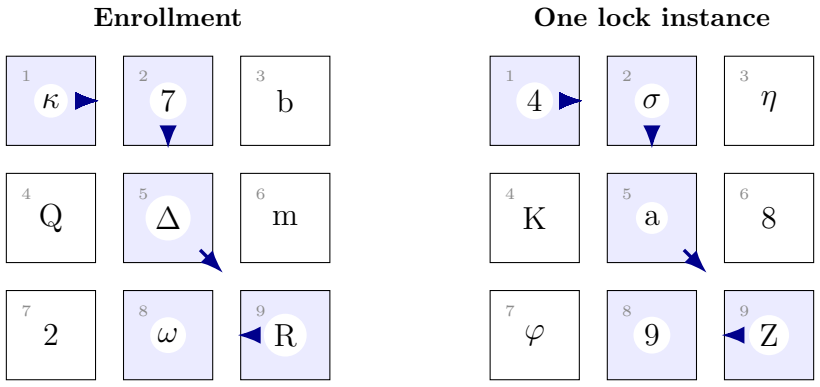


Figure 1: The same pattern on two coverings. Tinted cells are the layout pattern $1, 2, 5, 9, 8$. At enrollment the readout is $\kappa, 7, \Delta, \mathrm{R}, \omega$, and those contents generate the private key. In the lock instance the readout is $4, \sigma, \mathrm{a}, \mathrm{Z}, 9$, and the hash of that readout is the public hash.

SHA-256 of those five bytes is

`2da679a4f9135590f8b49b82ee1b728a536fa7c25375fbcd3cf3ab05b8c386e0`.

That digest is the public hash of this illustration. The layout writes 4 on cell 1, σ on cell 2, a on cell 5, Z on cell 9, and 9 on cell 8. Cells $3, 4, 6, 7$ receive decoys, here η, K, 8, and φ.

The owner repeats the stroke through cells $1, 2, 5, 9, 8$. The readout is again $4, \sigma, \mathrm{a}, \mathrm{Z}, 9$. Its SHA-256 equals the public hash, and the private key is released for the unwrap. The instance cache is wiped. The next presentation, including one that follows the display being turned off, uses a new nonce and a new public hash.

A 16-cell grid uses the same steps. Only n, the geometry of the pass-over rule, and the number of decoy cells change.

13 Threat model

The properties below are consequences of the mechanism. They are the threat model this specification meets.

A recorded public sequence dies with its instance.
An observer who learns the symbols a gesture read has learned that instance's public sequence. The sequence compares equal only to the public hash held beside the live nonce. When the display is turned off, or the unlock succeeds, that nonce and that public hash are wiped. Replaying the old symbol string against a later instance fails, because the later public hash was expanded from a new nonce.

A picture of the grid, without the gesture, does not reveal the seed.
The shown symbols are HKDF output. HKDF-SHA-256 does not yield the private key, and Argon2id does not yield the pin contents from the private key. The pin contents remain in the sealed record. The observer still has to produce a readout whose digest matches.

There is no standing numeric PIN to reuse.
The digits and letters on the glass are an instance covering. They are not the sealed pin contents, and they are not the private key.

Online guessing is guessing the gesture, and it is throttled.
Because the grid is arranged so that the stored pattern reads the public sequence, a guess is a guess of a stroke. A wrong stroke hashes to a different digest. The failure counter survives screen-off, so cycling the display does not refresh the attempt budget. The alphabet does not add a second secret the owner must remember, and it does not multiply the number of strokes an online attacker must try. Its job is to make each public reading one-time.

The private key is not the value on the glass.
The glass shows the public sequence and the decoys. The match authorizes one unwrap. The seed used for that unwrap is cleared from working memory with the instance.

The following exposures remain, and they follow from the same mechanism.

The gesture is durable.
The layout pattern is stored so that each new public sequence can be placed where the remembered stroke will read it. A person who learns that stroke, from watching the hand, from a recording, or from a smudge that still shows the path, can draw it on the next grid. The new grid will have been built for that path. Changing symbols does not retire a disclosed gesture.

The sealed record is the long-term secret.
A party who can read the hardware-backed record obtains the pin contents and the layout pattern, and can derive the private key with the stored salt and Argon2 parameters. The specification assumes that storage is the device's hardware-backed store and that ordinary software cannot read it.

A live instance is inside the trusted unlock path.
The component that draws the grid also sees the stroke on that grid. That is the same trust placed in the component that reads a PIN today. The instance cache is still wiped when the display turns off, so a later instance is a new public hash.

Pattern Pin removes reuse of an observed symbol string. It does this by generating each public hash from the private key and a nonce that dies with the lock instance. The gesture remains the thing the owner remembers, and the pin contents remain the thing the device stores in order to seed the private key.

14 Parameter summary

Parameter	Value
Grid	9 or 16 cells, one symbol each, chosen at enrollment
Alphabet	110 symbols, indexes 0 through 109, Section 3
Pattern length	4 through n distinct cells
Private key	Argon2id, 32 bytes, $m = 65536$ KiB, $t = 3$, $p = 4$, version `0x13`
Salt	16 bytes
Nonce	32 bytes, one lock instance, wiped with the instance
Public expansion	HKDF-SHA-256, info `pattern-pin/public-grid/v1`
Index sampling	Accept byte b when $b < 220$, emit $b \bmod 110$
Public hash	SHA-256 of the canonical public sequence
Check	Constant-time equality of that digest and the digest of the readout
Device-key wrap	AES-256-GCM, 12-byte nonce, 16-byte tag
Free failures	5, then 30 seconds doubling to a cap of 3600 seconds
Counter reset	Successful release only

15 Boundaries

Replacing a forgotten gesture is a new enrollment, which replaces the sealed record. This specification does not define a recovery channel around the sealed record. Sharing a lock across two people, unlocking a device remotely, and combining the gesture with a biometric are outside this mechanism. A 4×4 grid and a 3×3 grid are the two sizes this version defines.